Speeding Up Your Rust Developer Experience

Rustaceans Kenya

What you will learn

File: src/main.rs

```rust
fn main() {}

// A function that returns an `Option<Result<String, String>>`,
// for example a database operation that gets a username which
// returns an `Option<Result<Strin String>>` where an `Option::None`
// is returned if the username doesn't exist in the database or `Option::S
// is returned if the username exists and is valid.
// A `Result<String, String>` part is returned if an error or succe

// In this function, we return    `Result<Option<String>, Stri
// our database operation returns `Option<Result<String, St
// We can use our match statement to switch the `Result`
fn foo() -> Result<Option<String>, String> {
    let db_operation: Option<Result<String, String>

    match db_operation {
        Some(value) => match value {
            Ok(inner_value) => Ok(Some(inner_value)
            Err(error) => Err(error),
        },
        None => Ok(Option::None),
    }
}

// Or use `transpose` for cleaner code
fn foo() -> Result<Option<String>, String> {
    Database::get("username").transpose()
}
```

This event will improve your overall rust development experience. Learn caching methods, linkers and CDN techniques for slow networks .

UPDATE RUST TOOLCHAIN

Update the Rust tooling to the latest version. This tooling includes rustup, the rust compiler, cargo, rustfmt, clippy and more...

- Update all installed toolchains and tools in one go

```
$ rustup update
```

- Update only Rust stable version toolchains and tools

```
$ rustup update stable
```

- Update only Rust nightly version toolchains and tools

```
$ rustup update nightly
```

USE RUST-ANALYZER

Use Rust-Analyzer LSP plugin for your editor to elevate the experience to an IDE

- Learn how to install `rust-analyzer` plugin for yor favorite code editor or IDE

  https://rust-analyzer.github.io/

- Excerpt from their website:

```
rust-analyzer is an implementation of Language Server Protocol
for the Rust programming language. It provides features like
completion and goto definition for many code editors,
including VS Code, Emacs and Vim.
```

USE SCCACHE

Use sccache crate to cache built dependencies and improve speed on subsequent builds.

- In the global configuration `.cargo/config.toml` file where you installed the Rust toolchain, add this code. Replace `/path/to/sccache` with the actual path where `sccache` is installed.

```
[build]
rustc-wrapper = "/path/to/sccache"
```

FASTER LINKERS - LLD

A linker is a tool in the build process that takes compiled code (object files) and combines them into a final executable or library.

- Simplified linker pipeline

```
1. Source code is compiled into object files `(.rlib)`
2. Linker combines these `.rlib` files into the final executable
```

- A Rust .rlib file is a Rust-specific static library format that acts as an intermediate artifact during compilation.
- It contains compiled object code, LLVM bitcode for optimization, and essential Rust-specific metadata that the rustc compiler uses for linking.
- There are different linkers available for Rust but the default Rust toolchain now ships with LLD linker as of Rust 1.90.0.

FASTER LINKERS - MOLD

Mold is a faster drop-in replacement for existing Unix linkers. It is several times quicker than the default LLVM lld linker.

- After installing mold pass this to the `cargo` when building or checking

```
$ RUSTFLAGS="-Clink-arg=-fuse-ld=mold" cargo build
```

- For a better experience it is best to configure this globally in the `.cargo/config.toml` global Rust configuration file. This file can be found in the `.cargo` directory where you installed Rust:

```
[target.'cfg(target_os = "linux")']
rustflags = ["-C", "link-arg=-fuse-ld=mold"]
```

- If mold is installed in a custom path that is not visible to the Rust toolchain

```
[target.'cfg(target_os = "linux")']
rustflags = ["-C", "link-arg=-fuse-ld=/my/custom/path/to/mold"]
```

Mold is only available on Linux and Windows (x86_64 only) for download. Users on other operating systems might have to build from source.

FASTER LINKERS - WILD

Wild linker is a Rust native linker STILL UNDER DEVELOPMENT as of writing these slides

- After installing mold pass this to the `cargo` when building or checking. On Linux you can use `which wild` command to check the path it was intalled to.

```
RUSTFLAGS="-Clink-arg=-fuse-ld=/path/to/wild" cargo build
```

- For a better experience it is best to configure this globally in the `.cargo/config.toml` global Rust configuration file. This file can be found in the `.cargo` directory where you installed Rust:

```
[target.'cfg(target_os = "linux")']
rustflags = ["-C", "link-arg=-fuse-ld=/path/to/wild"]
```

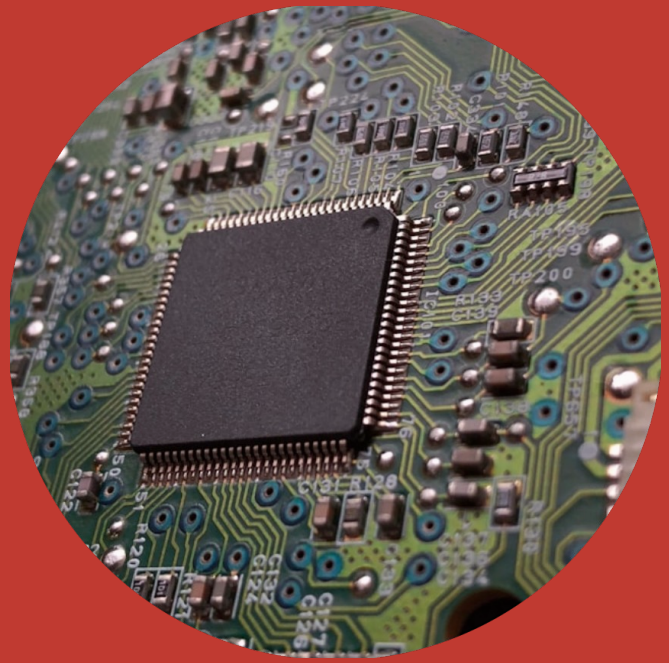
BACON - Compiling Automation

Bacon is a background code checker that is seen as a successor to `cargo-watch`. It automates checking code instead of repeating same tasks like cargo check

- Running `bacon` after installing.

```
$ bacon
```

- Read the docs for more usage information.



LIGHTER BUILD OPTIONS

Using `cargo build` can be slow since it compiles code to generate an executable. Use rust arguments that are lighter on the CPU.

- Use cargo check which only checks your code for correctness

```
$ cargo check
```

- `cargo clippy` works like cargo check but also shows helpful lints

```
$ cargo clippy
```

- Update the settings for you editor to use `cargo check` or `cargo clippy`. Eg. in vscode, press `ctrl+p` to open command palette and tap on `Open user settings JSON` and add the following line to the exisiting JSON

```
"rust-analyzer.check.command": "clippy",
```

CARGO OFFLINE

On a poor or disconnected internet connection, if you already fetched dependences for a previous project you can run `cargo offline` telling cargo to attempt using dependencies already cached in your `.cargo` global directory instead of fetching the index.

- This works just like any cargo command only with the `offline` flag attached. It only works for local dependencies so if a dependency is not cached it will not work

```
# To check correctness only
cargo check --offline

# To build
cargo build -offline

$ To run code
cargo run --offline
```

TRY OTHER CDNs OR A VPN

Rust toolchain install and updates downloads can be slow at times. Before attempting these solutions do a speed test to check if it's your connection that is actually slow. One way to tell is if other downloads are faster but Rust downloads are slow. I find on some Internet Service Providers, using a VPN can speed up downloading crates.io index.

Rust provides environment variables `RUSTUP_DIST_SERVER (for static binaries)` and `RUSTUP_UPDATE_ROOT` (for self-update). These are also useful for private repositories or permission networks like universities that provide their own servers.

- CDNs can also be faster. You can try any of these environment variables to install Rust toolchains. After `RUSTUP_DIST_SERVER=<url>` you can use any `rustup` command including `install` and `update`

```
RUSTUP_DIST_SERVER=https://cloudfront-static.rust-lang
.org  rustup install stable

RUSTUP_DIST_SERVER=https://fastly-static.rust-lang.org
rustup update
```

- If you live in China or plan to travel to China, ByteDance avails a high speed crates.io sparse index.

Learn more at https://rsproxy.cn/. Use Google translate :)

CUSTOM REGISTRIES

Cargo docs provide information on how to configure custom indexes to speed up fetching new crates when running `cargo run, cargo fetch, cargo check, etc`.

- Registries
- Source Replacement

SPLIT A LARGE PROJECT INTO A WORKSPACE

Splitting a large Rust project by using workspaces can result in faster build time since only the dependencies of the workspace member are rebuilt on subsequent builds

- Workspace file `Cargo,toml`

```
[workspace]
members = ["foobar"]
resolver = "3"

[workspace.package]
version = "0.1.0"
author = ["Example <support@example.com>"]
edition = "2024"

[workspace.dependencies]
```

The workspace member `foobar` is a crate in the workspace added manually or added using `cargo new` command

REMOVE UNUSED DEPENDENCIES

Some unused dependencies can pull in a lot of dependencies. Removing them can reduce compile times depending on unlinked dependencies

- A few tools are available to find unused dependencies
  - cargo-udeps
  - cargo-machete
  - cargo-shear
- `cargo-shear` has some more features like finding misplaced dependencies and unreachable source files because they are unlinked from module tree

CARGO TIMINGS

Find the crate that is taking up much of the compile times

- Cargo has a built-in feature for this

```
$ cargo build --timings
```

- The generated graphs are stored in the `target/cargo-timings/` directory in your current cargo workspace. The generated graph is in HTML format and can be open in your favorite web browser.

Learn more about compile times by reading the blog

intro-rustc-self-profile

CARGO LLVM-LINES

Find the functions that are taking too long to compile using llvm-lines crate

- Install `cargo-llvm-lines`

```
$ cargo install cargo-llvm-lines
```

- Example usage:

```
cargo llvm-lines | head -20
# … other compile time information
# ...
  Lines              Copies            Function name
  -----              ------            -------------
  80                 10                (TOTAL)
  24 (30.0%, 30.0%)  2 (20.0%, 20.0%)
core::ops::function::FnOnce::call_once
  12 (15.0%, 45.0%)  1 (10.0%, 30.0%) std::rt::lang_start
  10 (12.5%, 57.5%)  1 (10.0%, 40.0%)
core::fmt::Arguments::from_str
   9 (11.2%, 68.8%)  1 (10.0%, 50.0%) std::rt::lang_start::
{{closure}}
   7 (8.8%, 77.5%)   1 (10.0%, 60.0%) ...
std::process::Termination>::report
```

DISABLE UNUSED FEATURES

Only enable features you need

- Use `cargo add` to add dependencies to your project. It lists all features supported by the dependency you are adding.

```
cargo add tokio --no-default-features
    Updating crates.io index
      Adding tokio v1.50.0 to dependencies
             Features:
             - bytes
             - fs
             - full
            # more omitted features
```

- Now you can add the features you want, for example in tokio if you just want to use the async file system features you can enable the `fs` feature

```
cargo add tokio --no-default-features --features fs
    Updating crates.io index
      Adding tokio v1.50.0 to dependencies
             Features:
             + fs
             # .. Omitted diabled features ...
```

- You can also explore the cargo-features-manager crate to help you manage features of any dependency in your cargo workspace.

- Note that sometimes removing features does not improve compile times

TRY A DIFFERENT RUST COMPILER

Try cranelift Rust compiler for developer builds. Some workloads can receive a major speedup. It is only suitable for mostly developer workloads since it's still under development. Rust allows using LLVM for production builds

(cargo build --release)

The cranelift project

Other targets like WebAssembly can be faster to compile for especially procedural macro heavy code by using tools like Watt that compile for procedural macros

FURTHER READING

- Faster builds on MacOS
- MacOS: Exclude Rust Compilations from GateKeeper
- Windows 11 Dev Drive
- Tips For Faster Rust Compile Times

Rustaceans Kenya

Find slides and code for events at:
https://github.com/RustaceansKenya/Events